

Software Quality Assurance Process

Software Quality

- Software quality refers to the degree to which a software product meets the requirements and expectations of its stakeholders, including functionality, performance, maintainability, and usability.
- It encompasses both functional and non-functional aspects to ensure the software delivers value and operates reliably.
- Factors of software quality are:



Software Quality

Portability: A software is claimed to be transportable, if it may be simply created to figure in several package environments, in several machines, with alternative code merchandise, etc.

Usability: A software has smart usability if completely different classes of users (i.e. knowledgeable and novice users) will simply invoke the functions of the merchandise.

Reusability: A software has smart reusability if completely different modules of the merchandise will simply be reused to develop new merchandise.

Correctness: Software is correct if completely different needs as laid out in the SRS document are properly enforced.

Software Quality

Maintainability: A software is reparable, if errors may be simply corrected as and once they show up, new functions may be simply added to the merchandise, and therefore the functionalities of the merchandise may be simply changed, etc

Reliability: Software is more reliable if it has fewer failures. Since software engineers do not deliberately plan for their software to fail, reliability depends on the number and type of mistakes they make. Designers can improve reliability by ensuring the software is easy to implement and change, by testing it thoroughly, and also by ensuring that if failures occur, the system can handle them or can recover easily.

Software Quality

Efficiency. The more efficient software is, the less it uses of CPU-time, memory, disk space, [network bandwidth](#), and other resources. This is important to customers in order to reduce their costs of running the software, although with today's powerful computers, CPU time, memory and disk usage are less of a concern than in years gone by.

Software Reliability

- **Software reliability** is the probability that a software system will perform its intended functions without failure under specified conditions for a given period.
- It is a critical aspect of software quality, focusing on the system's dependability and consistent performance over time.

Software Quality Management and Planning

- **Software Quality Management (SQM)** is a systematic approach to ensure that a software product meets defined quality standards, user expectations, and business objectives.
- SQM encompasses processes, tools, and practices that focus on maintaining and improving the quality of software throughout its lifecycle.
- Quality management comprises the following activities: **quality control (QC), quality assurance (QA), and quality planning (QP).**

Software Quality Assurance(SQA)/ QA

- **Software Quality Assurance (SQA)** is simply a way to assure quality in the software.
- Software Quality Assurance is a process which works parallel to development of a software.
- It is the set of activities which ensure processes, procedures as well as standards suitable for the project and implemented correctly.
- It focuses on improving the process of development of software so that problems can be prevented before they become a major issue.

SQA Activities

1. **Creating an SQA Management Plan:** The foremost activity includes laying down a proper plan regarding how the SQA will be carried out in your project.
2. **Setting the Checkpoints:** The SQA team sets up different checkpoints according to which it evaluates the quality of the project activities at each checkpoint/project stage.
3. **Apply software Engineering Techniques:** Applying some software engineering techniques aids a software designer in achieving high-quality specification.
4. **Executing Formal Technical Reviews:** An FTR is done to evaluate the quality and design of the prototype.
5. **Having a Multi- Testing Strategy:** In multi-testing strategy one should not rely on any single testing approach, instead, multiple types of testing should be performed
6. **Measure Change Impact:** The changes for making the correction of an error sometimes re introduces more errors. So we need to properly measure the impact of change.

SQA Activities

7. **Performing SQA Audits:** The SQA audit inspects the entire actual SDLC process followed by comparing it against the established process. It also checks whatever reported by the team in the status reports were actually performed or not. This activity also exposes any non-compliance issues.
8. **Maintaining Records and Reports:** It is crucial to keep the necessary documentation related to SQA and share the required SQA information with the stakeholders. The test results, audit results, review reports, change requests documentation, etc. should be kept for future reference.
9. **Manage Good Relations:** In fact, it is very important to maintain harmony between the QA and the development team.

Quality Control(QC)

- Quality Control in Software Testing is a systematic set of processes used to ensure the quality of software products or services.
- The main purpose of the quality control process is ensuring that the software product meets the actual requirements by testing and reviewing its functional and non-functional requirements.

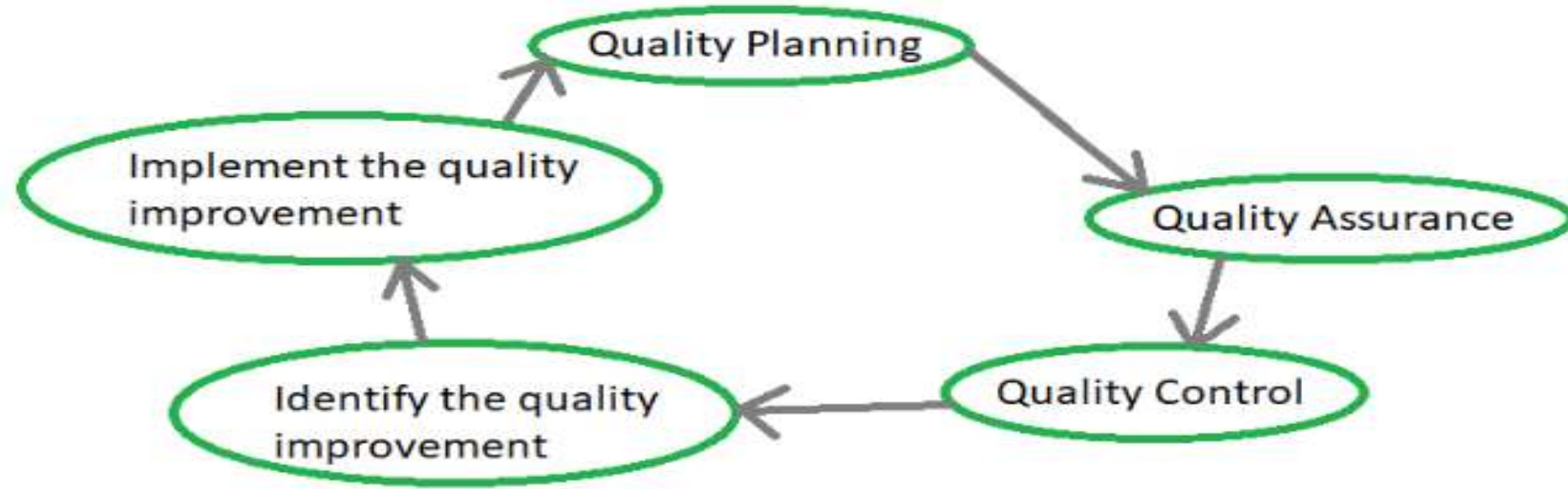
QA vs QC

QA	QC
It is a procedure that focuses on providing assurance that quality requested will be achieved	It is a procedure that focuses on fulfilling the quality requested.
QA aims to prevent the defect	QC aims to identify and fix defects
It is a method to manage the quality-Verification	It is a method to verify the quality-Validation
It does not involve executing the program	It always involves executing a program
It's a Preventive technique	It's a Corrective technique
QA involves in full software development life cycle	QC involves in full software testing life cycle
In order to meet the customer requirements, QA defines standards and methodologies	QC confirms that the standards are followed while working on the product

Quality Planning(QP)

- QP normally formalizes such aspects of quality management as:
 - Stakeholders' roles and responsibilities with regard to quality assurance.
 - Required project and testing documentation.
 - A software requirements specification and the set of standards software should comply with.
 - Relevant testing types.
 - An approximate testing schedule, cost, and human resources line-up for the corresponding project.
 - Training required by QA team members.
 - A reporting process and schedule.
 - A risk mitigation plan.

SQM process



Software Standards and Compliance

Software standards are established guidelines, rules, or specifications that software engineering processes, tools, and systems must adhere to. These standards ensure consistency, quality, and interoperability in software development and maintenance.

Types of Software Standards

1. ISO(International Organization for Standardization)
2. CMMI(Capability Maturity Model Integration)

Software Quality Standard- ISO Standards

ISO standards are internationally recognized guidelines, specifications, and frameworks developed by the **International Organization for Standardization (ISO)**. They provide consistent methods and criteria for ensuring quality, safety, efficiency, and interoperability across industries, products, services, and systems.

- An international set of standards for quality management
- ISO 9000 is defined as a set of international standards on quality management and quality assurance developed to help companies effectively document the quality system elements needed to maintain an efficient quality system.
- Applicable to a range of organisations from manufacturing to service industries

ISO 9000 is a series of three standards:



ISO Standard

- ISO 9001: *Model for quality assurance in design, development, production, installation, and servicing* was for companies and organizations whose activities included the creation of new products.
- ISO 9002: *Model for quality assurance in production, installation, and servicing* had basically the same material as ISO 9001 but without covering the creation of new products.
- ISO 9003: *Model for quality assurance in final inspection and test* covered only the final inspection of finished product, with no concern for how the product was produced.

CMMI

- The Capability Maturity Model Integration (CMMI) is an advanced framework designed to improve and integrate processes across various disciplines such as software engineering, systems engineering, and people management.
- It builds on the principles of the original CMM, enabling organizations to enhance their processes systematically.
- CMMI helps organizations fulfill customer needs, create value for investors, and improve product quality and market growth.
- It offers two representations, staged and continuous, to guide organizations in their process improvement efforts.

CMMI (Contd...)

Objectives of CMMI

- 1.Fulfilling customer needs and expectations.
- 2.Value creation for investors/stockholders.
- 3.Market growth is increased.
- 4.Improved quality of products and services.
- 5.Enhanced reputation in Industry.

CMMI (Contd...)

CMMI Representation – Staged and Continuous

A representation allows an organization to pursue a different set of improvement objectives. There are two representations for CMMI:

- **Staged Representation** :uses a pre-defined set of process areas to define improvement path.
 - provides a sequence of improvements, where each part in the sequence serves as a foundation for the next.
 - an improved path is defined by maturity level.
 - maturity level describes the maturity of processes in organization.
 - Staged CMMI representation allows comparison between different organizations for multiple maturity levels.

CMMI (Contd...)

- **Continuous Representation :**

- allows selection of specific process areas.
- uses capability levels that measures improvement of an individual process area.
- Continuous CMMI representation allows comparison between different organizations on a process-area-by-process-area basis.
- allows organizations to select processes which require more improvement.
- In this representation, order of improvement of various processes can be selected which allows the organizations to meet their objectives and eliminate [risks](#).

CMMI (Contd...)

CMMI Model – Maturity Levels

In CMMI with staged representation, there are five maturity levels described as follows :

1. Maturity level 1 : Initial

- processes are poorly managed or controlled.
- unpredictable outcomes of processes involved.
- ad hoc and chaotic approach used.
- No KPAs (Key Process Areas) defined.
- Lowest quality and highest risk.

CMMI (Contd...)

CMMI Model – Maturity Levels

2. Maturity level 2 : Managed

- requirements are managed.
- processes are planned and controlled.
- projects are managed and implemented according to their documented plans.
- This risk involved is lower than Initial level, but still exists.
- Quality is better than Initial level.

CMMI (Contd...)

CMMI Model – Maturity Levels

3. Maturity level 3 : Defined

- processes are well characterized and described using standards, proper procedures, and methods, tools, etc.
- Medium quality and medium risk involved.
- Focus is process standardization.

CMMI (Contd...)

CMMI Model – Maturity Levels

4. Maturity level 4 : Quantitatively managed

- quantitative objectives for process performance and quality are set.
- quantitative objectives are based on customer requirements, organization needs, etc.
- process performance measures are analyzed quantitatively.
- higher quality of processes is achieved.
- lower risk

CMMI (Contd...)

CMMI Model – Maturity Levels

5.Maturity level 5 : Optimizing

- continuous improvement in processes and their performance.
- improvement has to be both incremental and innovative.
- highest quality of processes.
- lowest risk in processes and their performance.

CMMI (Contd...)

CMMI Model – Capability Levels

A capability level includes relevant specific and generic practices for a specific process area that can improve the organization's processes associated with that process area. For CMMI models with continuous representation, there are six capability levels as described below :

1.Capability level 0 : Incomplete

- incomplete process – partially or not performed.
- one or more specific goals of process area are not met.
- No generic goals are specified for this level.
- this capability level is same as maturity level 1.

CMMI (Contd...)

CMMI Model – Capability Levels

2.Capability level 1 : Performed

- process performance may not be stable.
- objectives of quality, cost and schedule may not be met.
- a capability level 1 process is expected to perform all specific and generic practices for this level.
- only a start-step for process improvement.

CMMI (Contd...)

CMMI Model – Capability Levels

3. Capability level 2 : Managed

- process is planned, monitored and controlled.
- managing the process by ensuring that objectives are achieved.
- objectives are both model and other including cost, quality, schedule.
- actively managing processing with the help of metrics

CMMI (Contd...)

CMMI Model – Capability Levels

4.Capability level 3 : Defined

- a defined process is managed and meets the organization's set of guidelines and standards.
- focus is process standardization.

5.Capability level 4 : Quantitatively Managed

- process is controlled using statistical and quantitative techniques.
- process performance and quality is understood in statistical terms and metrics.
- quantitative objectives for process quality and performance are established.

CMMI (Contd...)

CMMI Model – Capability Levels

6. Capability level 5 : Optimizing

- focuses on continually improving process performance.
- performance is improved in both ways – incremental and innovation.
- emphasizes on studying the performance results across the organization to ensure that common causes or issues are identified and fixed

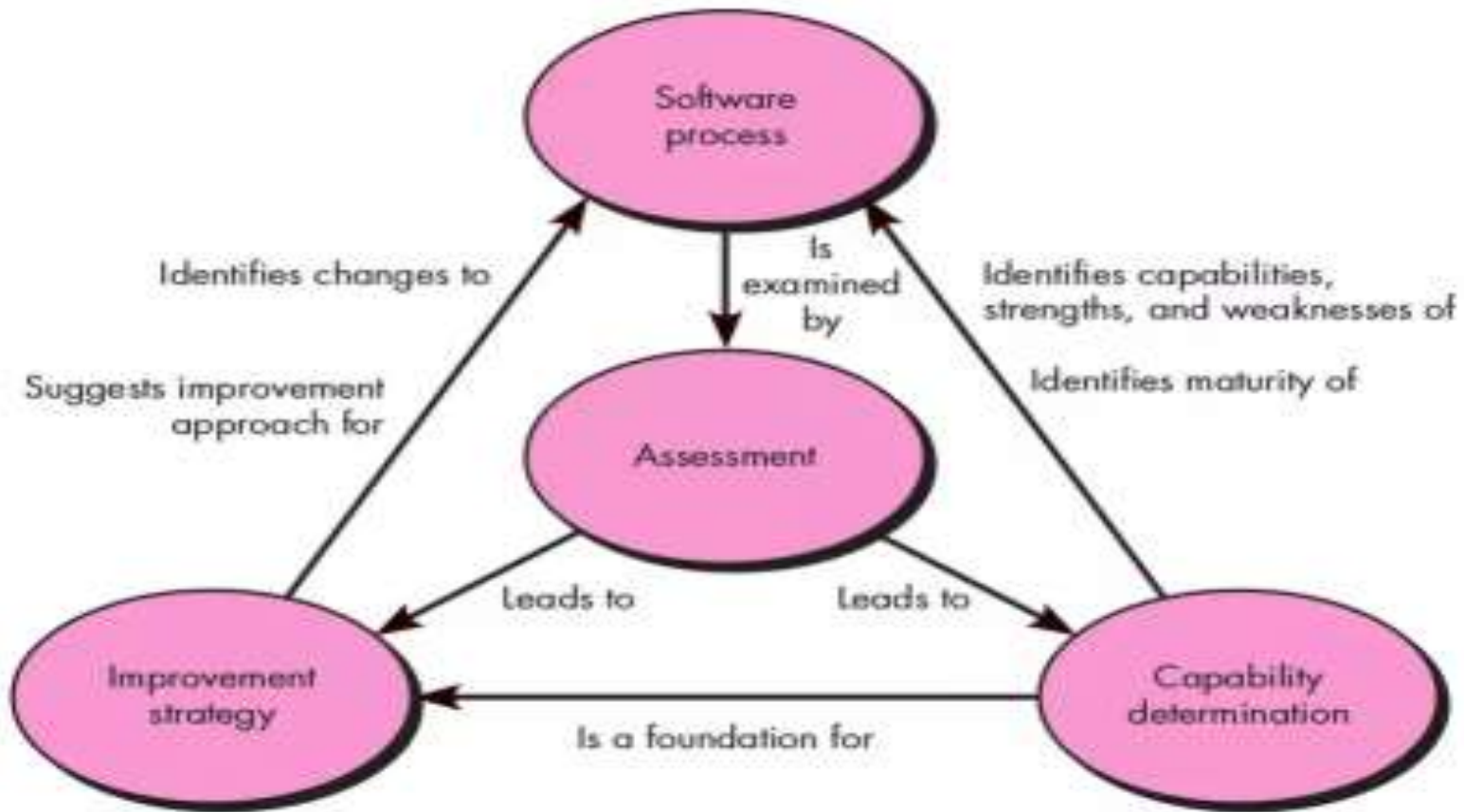
Capability Assessment and Process Improvement

Capability assessment in software engineering evaluates an organization's processes, practices, and capabilities to determine their effectiveness in producing high-quality software. It identifies strengths, weaknesses, and areas for improvement within the organization's software development lifecycle (SDLC).

Process improvement in software engineering refers to the systematic practice of analyzing, optimizing, and enhancing software development processes to achieve better efficiency, quality, and predictability. It focuses on reducing defects, increasing productivity, and ensuring timely delivery of software products.

Capability Assessment and Process Improvement

Elements of an SPI Framework



Software Inspection

- Software Inspection is a static process.
- Inspection is necessary to review a product
- Inspection in software engineering, refers to peer review of any work product by trained individuals who look for defects using a well defined process..
- In an inspection process, the objective is to identify defects, and the inspected parts are generally SRS(Software Requirements Specifications) documents and test plans.

Need of Software Inspection

- Reduced development and maintenance costs
- Improved software quality and reliability
- Initiate effective verification and validation
- Reduced cost and risk of software testing
- Reduced dependence on quality assurance
- Support software quality metrics, process metrics, defect prevention, change management, etc

Software Inspection-Types

1. Code Review

- The code review can be described as a process of examination/inspection in which a piece or block of code is treated as a defect.
- The code is a defect if it does not deliver the right functionality, or may be sometimes the code could be improved in terms of its performance.

2. Peer Review

- Peer means group, so in this kind of inspection a group of people who have requisite knowledge and skills, review the product.
- An author asks a peer to read, comment, and critique a work artifact.
- If the work artifact is code, the reviewer will read the code, and may even develop and run some unit tests to check that the code works as advertised.

.

Inspection Roles

1. **Author:** Author is the person who has created the work product, that is to be inspected.
2. **Moderator:** Moderator is the leader of the inspection.
3. **Reader:** Reader is the person responsible for reading the documents.
4. **Recorder/Scribe:** This person is the one who records the defects that are being discovered.
5. **Inspector:** Responsible for examining the product and identify possible defects.

Phases In Inspection Process

1. **Planning:** In this phase the moderator plans the activities to be performed during inspection.
2. **Overview Meeting:** An overview meeting intends to disseminate information regarding the background of the product under review.
3. **Preparation:** Preparation is a major step towards final accomplishment. In this phase we identify possibility of occurrence of a certain defect.
4. **Inspection meeting:** During this phase, the reader reads through the product and the inspector tries to point out the defects.
5. **Rework:** Based on the inspection meeting, the moderator makes necessary changes to the product.
6. **Follow up:** In follow up, The changes made are thus reviewed by the author.

Benefits of Inspection

1. During testing, error can mask(hide) other errors. Since inspection is a static process, we don't have to be concerned with interactions between errors.
2. **Knowledge Sharing** - A person who has fair share of knowledge can spread it further regarding design specifications and specific software artefacts and also defect detection methods.
3. **Early fault finding** - Early detection definitely proves to be an advantage always. In this case also prior detection helps in cost reduction. Detection may be during the detailed design phase or requirements phase.